



Dokumentacja projektu

2025/2026

Decipher

Projekt – Bazy Danych

Kierunek: Informatyka

Członkowie zespołu:

1. Adam Drażyk
2. Karol Borowski
3. Piotr Hejmo

Gliwice, 24 września 2026

Spis treści

1	Założenia projektu	2
1.1	Cel projektu	2
1.2	Role w projekcie	3
1.3	Encje	3
1.3.1	Relacje między encjami	3
2	Budowa projektu	5
2.1	Model relacyjny	5
2.1.1	Tabele słownikowe	5
2.1.2	Tabele transakcyjne	6
2.1.3	Tabele asocjacyjne	6
2.2	Procedura	6
2.3	Interakcja z bazą danych	8
2.3.1	Kluczowe założenia	8
2.3.2	Podział funkcjonalny metod	9
3	Podsumowanie projektu	11
3.1	Działanie aplikacji	11
3.2	Konta użytkownika	17
3.3	Podsumowanie i wnioski	17

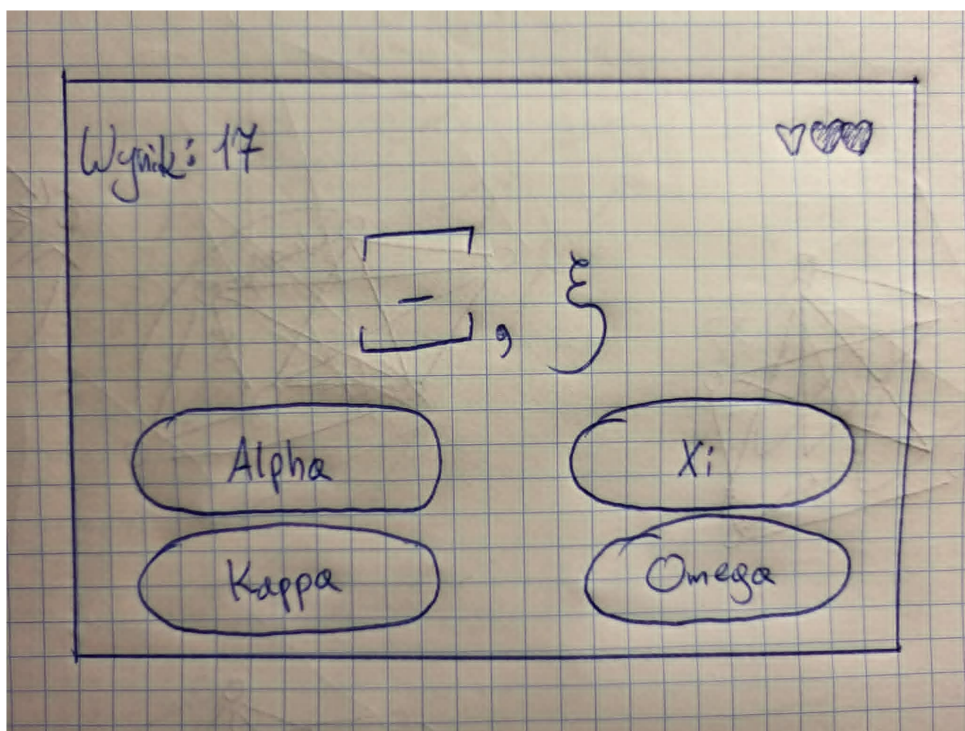
Rozdział 1

Założenia projektu

1.1 Cel projektu

Celem projektu było stworzenie aplikacji desktopowej (nazwanej z czasem *Decipher*), służącej do nauki różnych alfabetów oraz innych fonetycznych systemów pisma. Aplikacja miałaby działać na podstawie rozbudowanej bazy danych zawierającej liczne systemy pisma wraz z powiązanymi z nimi literami.

Jednym z głównych celów przy tworzeniu aplikacji było zaimplementowanie mechanik grywalizacji, takich jak system punktów życia czy możliwość pobijania najlepszych wyników, co miało by poskutkować większym zaangażowaniem oraz przyjemnością z nauki nowych systemów pisma.



Rysunek 1.1: Pierwszy szkic głównego widoku gry

Aplikacja miała zostać napisana w języku C# zgodnie z wzorcem architektonicznym MVP, w środowisku Visual Studio w Windows Forms .NET. Część bazodanowa miała być przygotowana w MySQL oraz hostowana lokalnie na komputerze użytkownika.

1.2 Role w projekcie

Grupa tworząca projekt składała się z trzech osób:

- **Adam Drażyk** – lider zespołu, odpowiedzialny za koordynację pracy, przygotowanie widoków, interfejsów oraz danych do tabel słownikowych.
- **Karol Borowski** – odpowiedzialny za działanie aplikacji, tworzenie prezenterów oraz modeli, a także implementację mechanizmów grywalizacji
- **Piotr Hejmo** – odpowiedzialny za część bazodanową, przygotowanie relacyjnej bazy danych oraz poprawne połączenie jej z aplikacją poprzez plik `Repository.cs`.

1.3 Encje

W systemie wyróżniono cztery główne encje, z których każda opisana jest unikalnym identyfikatorem (ID) oraz zestawem atrybutów:

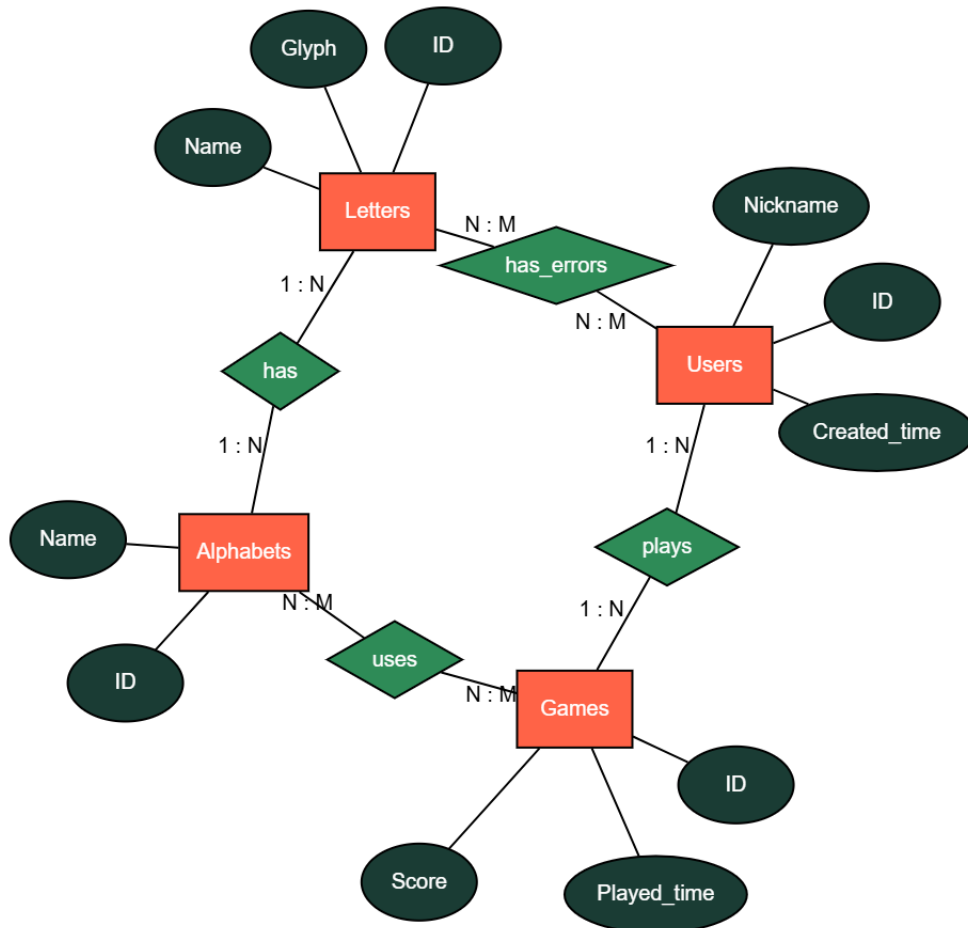
- **Users** – Reprezentuje graczy korzystających z aplikacji. Encja przechowuje unikalny pseudonim gracza (`Nickname`) oraz moment utworzenia profilu (`Created_time`).
- **Games** – Odzwierciedla pojedynczą sesję rozgrywki. Przechowuje informacje o końcowym wyniku punktowym (`Score`) uzyskanym przez gracza oraz dacie zakończenia partii (`Played_time`).
- **Alphabets** – Definiuje systemy pisma dostępne jako kategorie w grze. Jej głównym atrybutem jest nazwa własna alfabetu (`Name`).
- **Letters** – Reprezentuje pojedyncze znaki testowane w quizach. Każdy znak posiada swoją graficzną reprezentację, czyli glif (`Glyph`), oraz przypisaną poprawną odpowiedź, czyli nazwę (`Name`).

1.3.1 Relacje między encjami

Interakcje pomiędzy encjami definiują przepływ procesów i mechanikę gry w aplikacji.

- **plays** – Relacja typu jeden-do-wielu $1 : N$ między encjami **Users** i **Games**. Jeden użytkownik może rozegrać wiele gier, natomiast każda rozegrana gra jest jednoznacznie przypisana tylko do jednego gracza.
- **uses** – Relacja typu wiele-do-wielu $N : M$ między encjami **Games** i **Alphabets**. W ramach jednej gry gracz może być testowany z wiedzy o wielu alfabetach jednocześnie (tryb mieszany), a dany alfabet może stanowić element wielu różnych gier.

- **has** – Relacja typu jeden-do-wielu $1 : N$ między encjami **Alphabets** i **Letters**. Każdy alfabet składa się z wielu liter, a każda przypisana litera przynależy strukturalnie do konkretnego systemu pisma.
- **has_errors** – Relacja typu wiele-do-wielu $N : M$ między encjami **Users** i **Letters**. Użytkownik w trakcie korzystania z aplikacji może pomylić się przy wielu różnych znakach, a konkretna litera może sprawiać trudność wielu różnym graczom.



Rozdział 2

Budowa projektu

2.1 Model relacyjny

Architektura bazy danych została zaprojektowana w sposób relacyjny. Struktura została znormalizowana i podzielona na trzy logiczne grupy: tabele słownikowe (przechowujące stałe dane referencyjne), tabele transakcyjne (rejestrujące zdarzenia i aktywność w systemie) oraz tabele asocjacyjne (realizujące powiązania typu wiele-do-wielu).

2.1.1 Tabele słownikowe

Tabele z tej grupy stanowią zbiór danych referencyjnych, do których odwołują się inne elementy systemu.

- **alphabets** – Tabela przechowująca zdefiniowane w aplikacji systemy pisma. Kluczem głównym jest atrybut `id` (`int`). Nazwa alfabetu przechowywana jest w kolumnie `name` (`varchar(100)`).

id	name
1	Latin
2	Cyrillic
⋮	⋮

- **letters** – Tabela przechowująca pojedyncze znaki przypisane do poszczególnych systemów pisma. Kluczem głównym jest `id` (`int`). Reprezentacja graficzna znaku znajduje się w kolumnie `glyph` (`varchar(50)`), a jego pełna nazwa w `name` (`varchar(100)`). Relacja z tabelą nadrzędną realizowana jest poprzez opcjonalny klucz obcy `alphabet_id` (`int`).

id	glyph	name	alphabet_id
3	A, a	A	1
4	B, b	Bee	1
4	C, c	Cee	1
4	B, d	Dee	1
⋮	⋮	⋮	⋮

2.1.2 Tabele transakcyjne

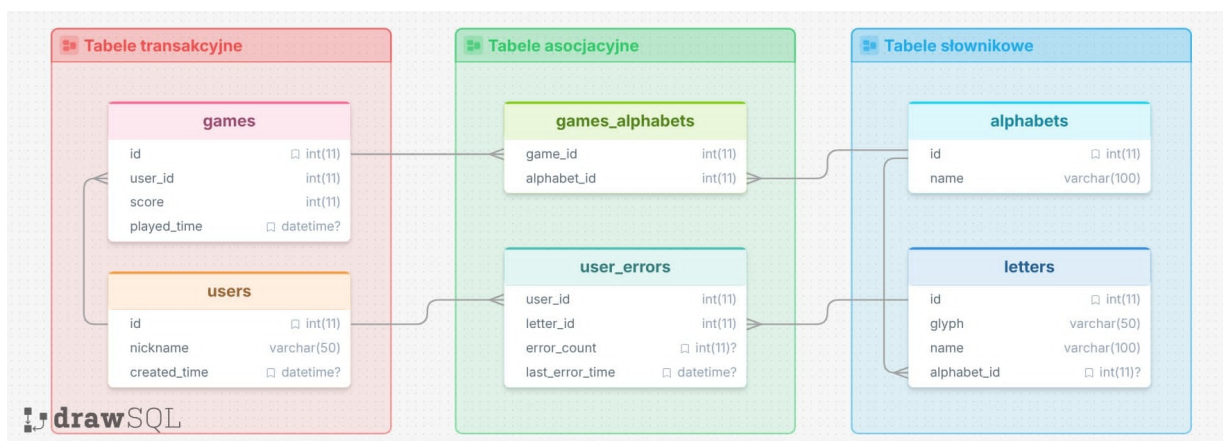
Grupa tabel odpowiedzialna za przechowywanie faktów operacyjnych oraz tworzenie głównego strumienia danych w aplikacji.

- **users** – Centralna tabela przechowująca konta graczy, identyfikowana przez klucz główny `id` (`int`). Zawiera unikalny identyfikator wizualny gracza `nickname` (`varchar(50)`) oraz znacznik czasu utworzenia konta `created_time` (`datetime`).
- **games** – Tabela rejestrująca zsumowane dane z pojedynczych sesji rozgrywki. Kluczem głównym jest `id` (`int`). Informacja o tym, kto rozegrał partię, zapewniana jest przez klucz obcy `user_id` (`int`). Ponadto tabela przechowuje zdobyty wynik punktowy `score` (`int`) oraz datę zakończenia gry `played_time` (`datetime`).

2.1.3 Tabele asocjacyjne

Grupa odpowiedzialna za rozbijanie złożonych relacji wiele-do-wielu $M : N$ na dwie relacje jeden-do-wielu $1 : N$, gwarantując integralność referencyjną i zapobiegając redundancji.

- **games_alphabets** – Tabela realizująca relację przypisania wielu alfabetów do jednej gry oraz wielu gier do konkretnych alfabetów. Składa się wyłącznie z dwóch kluczy obcych: `game_id` (`int`) oraz `alphabet_id` (`int`).
- **user_errors** – Tabela asocjacyjna zawierająca atrybuty opisujące. Łączy użytkowników z literami, rejestrując indywidualne błędy. Składa się z kluczy obcych `user_id` (`int`) i `letter_id` (`int`) oraz kolumn śledzących statystyki: łącznej liczby pomyłek dla danego znaku `error_count` (`int`) i czasu wystąpienia ostatniego błędu `last_error_time` (`datetime`).



2.2 Procedura

Procedura **BESTSCORE** służy do przetwarzania surowych danych relacyjnych w celu zwrócenia listy trzynastu najlepszych wyników osiągniętych przez konkretnego użytkownika. Agregacja odbywa się na poziomie unikalnych kombinacji alfabetów, co oznacza, że z wielu gier rozegranych w tej samej kombinacji wybierany jest wyłącznie najwyższy wynik.

Algorithm 1: Procedura BESTSCORE

Input: Identyfikator użytkownika p_UserId

Output: Lista 13 najlepszych wyników dla unikalnych konfiguracji alfabetów

```
1  $R \leftarrow \sigma_{user\_id=p\_UserId}(games) \bowtie games\_alphabets \bowtie alphabets$ 
2  $GamesGrouped \leftarrow GROUP\_BY(R, game\_id)$ 
3  $TempList \leftarrow \emptyset$ 
4 foreach  $g \in GamesGrouped$  do
5    $CombinedAlphabets \leftarrow \sum g.alphabet\_name_i$ 
6    $score \leftarrow g.score$ 
7    $TempList \leftarrow TempList \cup \{(CombinedAlphabets, score)\}$ 
8 end
9  $FinalResults \leftarrow \emptyset$ 
10  $Categories \leftarrow UNIQUE(CombinedAlphabets)$ 
11 foreach  $c \in Categories$  do
12    $MaxScore \leftarrow MAX(\{s : (c, s) \in TempList\})$ 
13    $FinalResults \leftarrow FinalResults \cup \{(c, MaxScore)\}$ 
14 end
15  $SORT(FinalResults, MaxScore, DESC)$ 
16 return  $LIMIT(FinalResults, 13)$ 
```

Metoda `GetUserBestScores` pełni rolę warstwy dostępu do danych. Jej głównym zadaniem jest nawiązanie połączenia z bazą danych MySQL, wywołanie zdefiniowanej wcześniej procedury składowanej o nazwie `BESTSCORE` oraz zmapowanie zwróconych, list obiektów `BestScores`.

Omówienie działania kodu 2.1:

- **Zarządzanie zasobami:** Wykorzystanie instrukcji `using` dla obiektów `MySqlConnection`, `MySqlCommand` oraz `MySqlDataReader` gwarantuje automatyczne zamknięcie połączenia po opuszczeniu bloku kodu.
- **Przekazywanie parametrów:** Zastosowanie metody `Parameters.AddWithValue` do przekazania identyfikatora użytkownika chroni aplikację przed atakami typu SQL Injection. Wartość jest automatycznie typowana przez `MySqlConnector`, zamiast być bezpośrednio doklejana do ciągu znaków zapytania.
- **Odczyt i mapowanie danych:** Wywołanie `ExecuteReader` otwiera jednokierunkowy strumień danych. Pętla `while (reader.Read())` iteruje po każdym zwróconym wierszu z bazy. Następuje tutaj proces mapowania – surowe dane wyciągnięte za pomocą ich nazw kolumn (`GetString("name")`, `GetInt32("MaxScore")`) są przypisywane do właściwości nowych instancji klasy `BestScores`, które następnie trafiają do zwracanej listy.

```
1 public List<BestScores> GetUserBestScores(int userId)
2 {
3     var scores = new List<BestScores>();
4
5     using (var connection = new MySqlConnection(_connection))
6     {
7         connection.Open();
8
9         using (var command = new MySqlCommand("BESTSCORE", connection))
```

```

10     {
11         command.CommandType = System.Data.CommandType.
            StoredProcedure;
12
13         command.Parameters.AddWithValue("p_UserId", userId);
14
15         using (var reader = command.ExecuteReader())
16         {
17             while (reader.Read())
18             {
19                 scores.Add(new BestScores
20                 {
21                     Alphabet = reader.GetString("name"),
22                     Score = reader.GetInt32("MaxScore")
23                 });
24             }
25         }
26     }
27 }
28
29 return scores;
30 }

```

Listing 2.1: Implementacja metody pobierającej najlepsze wyniki gracza

2.3 Interakcja z bazą danych

Klasa `Repository` pełni w aplikacji rolę Warstwy Dostępu do Danych. Stanowi ona fizyczną implementację interfejsu `IRepository`. Jej głównym zadaniem jest separacja logiki biznesowej i prezentacyjnej od zapytań SQL oraz szczegółów technicznych silnika bazy danych MySQL.

2.3.1 Kluczowe założenia

Architektura tej klasy opiera się na trzech głównych filarach, gwarantujących stabilność i bezpieczeństwo aplikacji:

- **Zarządzanie zasobami:** Wszystkie połączenia (`MySqlConnection`), komendy (`MySqlCommand`) oraz obiekty czytające (`MySqlDataReader`) są zamykane w blokach `using`. Gwarantuje to bezpieczne zarządzanie pamięcią niezarządzaną oraz automatyczne zamykanie połączeń z bazą, co zapobiega wyczerpaniu puli połączeń.
- **Ochrona przed SQL Injection:** Klasa unika konkatencji ciągów tekstowych przy przekazywaniu danych od użytkownika. Zamiast tego wykorzystuje mechanizm parametryzacji zapytań (np. `Parameters.AddWithValue`), co neutralizuje ryzyko ataków typu SQL Injection.
- **Mapowanie Obiektowo-Relacyjne:** Klasa pełni rolę ORM-a. Surowe dane pobierane z tabel relacyjnych są natychmiast mapowane na struktury języka C# (np. obiekty `Letter`, `BestScores`, `WeakestLetter` lub kolekcje `List` i `Dictionary`), co znacznie ułatwia pracę w pozostałych warstwach aplikacji.

2.3.2 Podział funkcjonalny metod

Logikę klasy Repository można podzielić na cztery główne obszary operacyjne:

Zarządzanie Użytkownikami

Grupa metod odpowiadająca za identyfikację oraz tworzenie nowych profili graczy:

- `CreateNewPlayer(string nickname)`: Realizuje podstawową operację wstawienia INSERT nowego użytkownika do tabeli `users`.
- `GetAllPlayerNames()`: Pobiera listę wszystkich dostępnych pseudonimów z bazy, zwracając je jako `List<string>`, co zasila interfejs wyboru profilu.
- `GetUserIdByNickname(string nickname)`: Zoptymalizowana metoda wyszukująca identyfikator gracza. Wykorzystuje klauzulę `LIMIT 1` oraz wywołanie `ExecuteScalar()` do pobierania pojedynczej wartości z pierwszego wiersza i kolumny wyniku.

Pobieranie Danych Gry

Metody odpowiedzialne za dostarczenie konfiguracji oraz materiału do silnika quizu:

- `GetAvailableAlphabets()`: Zwraca słownik (`Dictionary<int, string>`), gdzie kluczem jest identyfikator alfabetu, a wartością jego nazwa.
- `GetAllLetters()`: Pobiera wszystkie litery ze słownika. Radzi sobie z potencjalnym brakiem przypisanego alfabetu za pomocą metody `IsDBNull()`.
- `GetLettersByAlphabetIds(List<int> alphabetIds)`: Zaawansowana metoda dynamicznie generująca klauzulę `IN (·)` w zapytaniu SQL. Metoda w pętli generuje unikalne aliasy parametrów (np. `@id0`, `@id1`), chroniąc aplikację przed błędami składni.

Zapis Wyników i Transakcje

Obszar krytyczny pod kątem spójności danych:

- `SaveGameScore(int userId, int score, List<int> alphabetIds)`: Zapisuje historię rozgrywki wykorzystując **Transakcje Bazy Danych** (`BeginTransaction`). W jednym bloku transakcyjnym rejestruje nową grę, pobiera jej identyfikator (`SELECT LAST_INSERT_ID()`), a następnie iteruje po liście wybranych alfabetów, tworząc relacje w tabeli `games_alphabets`. W przypadku błędu instrukcja `transaction.Rollback()` cofa wszystkie zmiany.
- `RecordUserError(int userId, int letterId)`: Rejestruje pomyłki gracza wykorzystując konstrukcję `ON DUPLICATE KEY UPDATE`. W przypadku pierwszej pomyłki tworzy rekord, a przy kolejnych jedynie inkrementuje licznik błędów.

Agregacja Statystyk

Metody wyciągające analityczne podsumowania dla widoku profilu gracza:

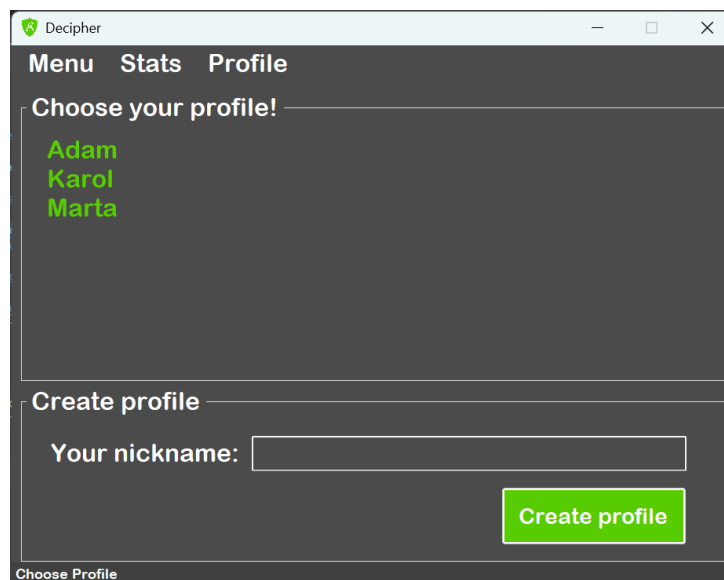
- `GetUserBestScores(int userId)`: Deleguje operacje grupowania ciągów znaków i szukania wartości maksymalnych do serwera bazy danych, wywołując procedurę `BESTSCORE`.
- `GetUserBestLanguages(int userId)`: Pobiera najlepsze wyniki dla pojedynczych alfabetów, ignorując tryby mieszane za pomocą klauzuli `HAVING COUNT(alphabet_id) = 1`.
- `GetUserWeakestLetters(int userId)`: Przeprowadza złączenie JOIN rejestru błędów użytkownika ze słownikiem liter, sortując wyniki malejąco i zwracając listę najczęściej mylonych znaków.

Rozdział 3

Podsumowanie projektu

3.1 Działanie aplikacji

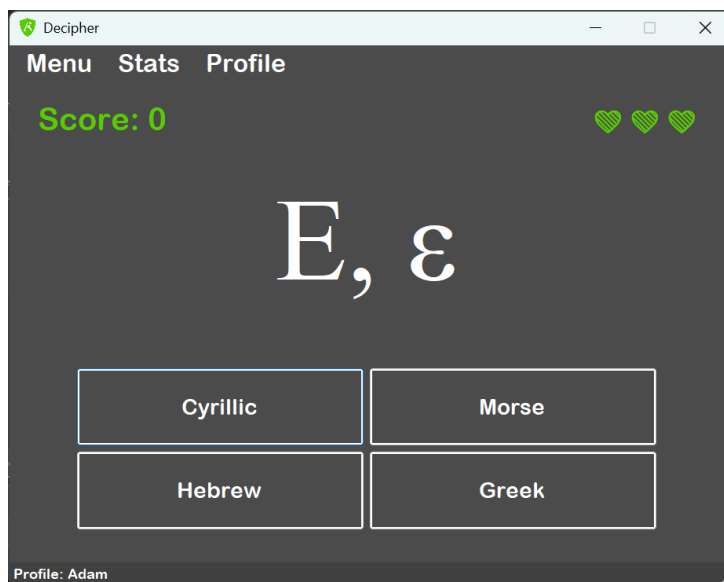
Po uruchomieniu aplikacji użytkownik natrafia na menu wyboru profilu 3.1, gdzie użytkownik ma możliwość stworzenia profilu bądź wyboru już istniejącego.



Rysunek 3.1: Menu wyboru profilu

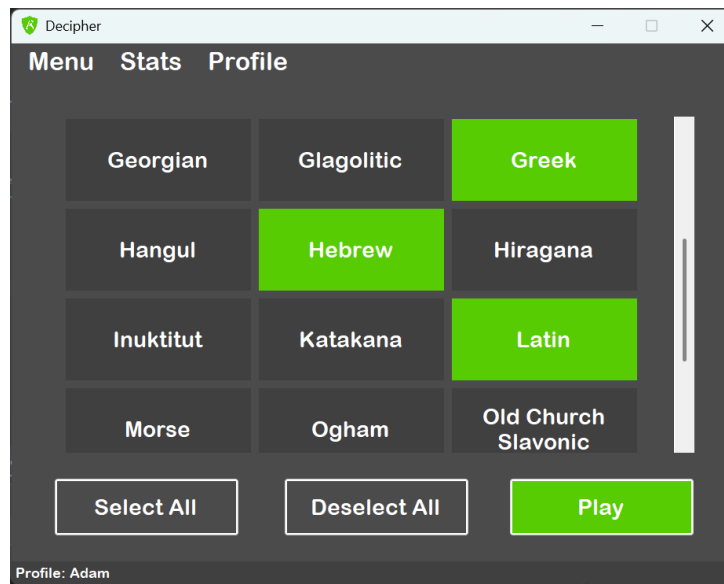
Po wybraniu profilu użytkownik zostaje przeniesiony do menu głównego ??, gdzie do wyboru ma jeden z dwóch trybów gry:

- **Guess the Script** w tym trybie gracz na podstawie litery (glifu) musi poprawnie wskazać jeden z czterech wylosowanych alfabetów, do którego wyświetlona litera należy 3.2.



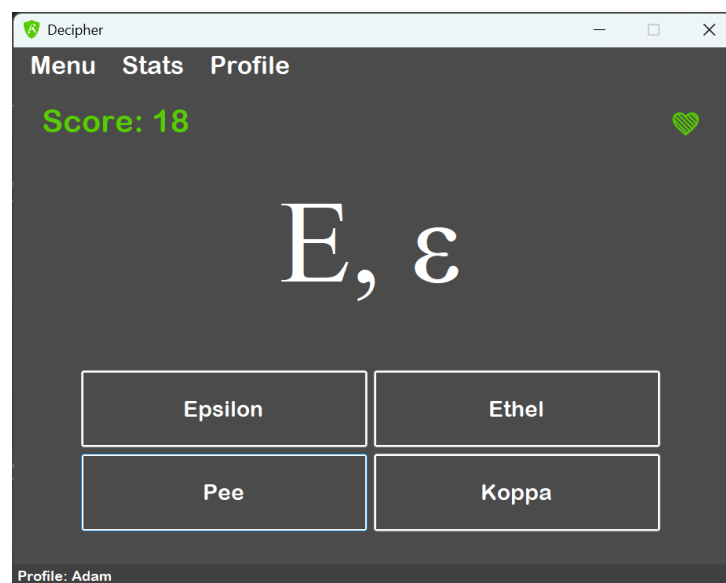
Rysunek 3.2: Tryb gry: **Guess the Script**

- **Guess the Letter** po wyboru tego trybu gracz zostaje przeniesiony do widoku wyboru alfabetów 3.3, w którym wybiera, z których spośród nich będzie chciał rozpoznawać litery. Wybór alfabetu odbywa się poprzez kliknięcie PPM na dany przycisk.



Rysunek 3.3: Menu wyboru alfabetów

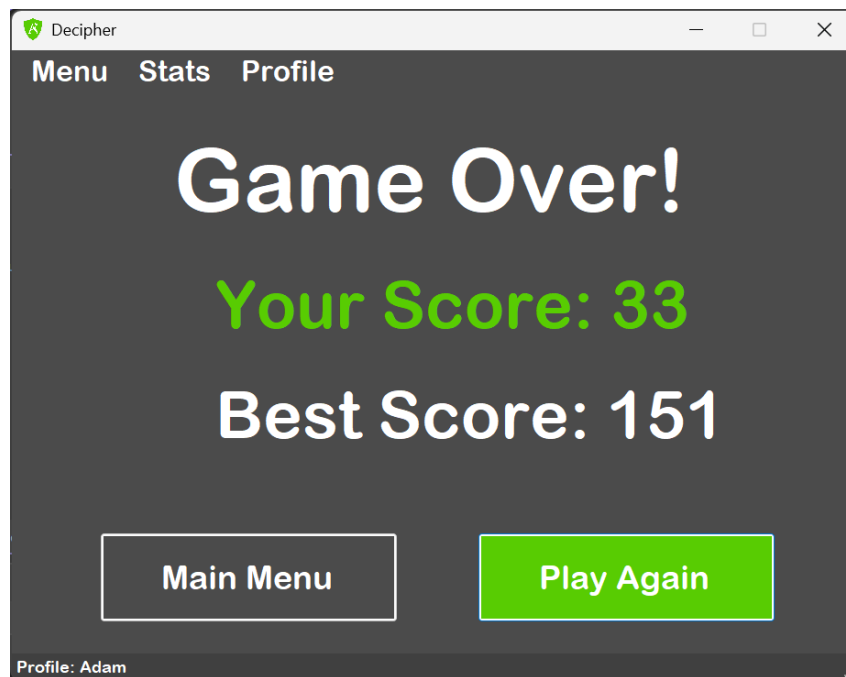
Po wciśnięciu przycisku Play rozpoczyna się gra 3.4, widok wygląda identycznie jak w przypadku trybu **Guess the Script** jednak zadaniem gracza jest poprawne dopasowanie nazwy danej litery (wersja angielska) do glifu wyświetlanego powyżej.



Rysunek 3.4: Tryb gry: Guess the Letter

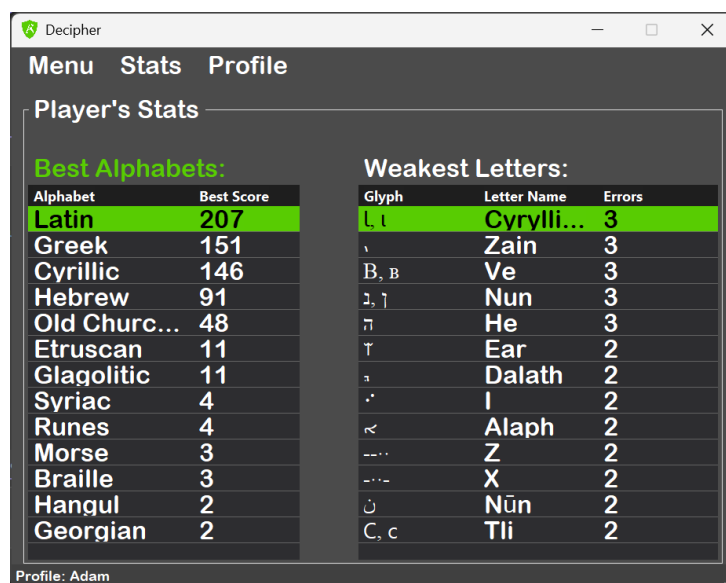
W lewym górnym rogu widoku gracz może zobaczyć wynik **Score**, który informuje użytkownika ilu poprawnych odpowiedzi udzielił. W prawym górnym rogu gracz widzi natomiast ile jeszcze pomyłek mu zostało. Liczba „życ” gracza reprezentowana jest liczbę serc ♥.

Gra kończy się w momencie, w którym gracz utraci wszystkie punkty życia. Wyświetlony zostaje wtedy widok końca gry, na którym gracz ma możliwość porównania swojego wyniku z najlepszym wynikiem (na tej samej konfiguracji trybu gry, uruchomienia gry ponownie bądź powrotu do menu głównego).



Rysunek 3.5: Widok Game Over

Ostatnim widokiem jest widok statystyk 3.6, można go otworzyć wybierając odpowiednią opcję w menu. W widoku tym użytkownik może zobaczyć jak idą mu konkretne alfabety oraz z jakimi literami ma największy problem.



Decipher

Menu Stats Profile

Player's Stats

Best Alphabets:

Alphabet	Best Score
Latin	207
Greek	151
Cyrillic	146
Hebrew	91
Old Churc...	48
Etruscan	11
Glagolitic	11
Syriac	4
Runes	4
Morse	3
Braille	3
Hangul	2
Georgian	2

Weakest Letters:

Glyph	Letter Name	Errors
Ł, ł	Cyrylli...	3
ʾ	Zain	3
В, в	Ve	3
נ, ן	Nun	3
ה	He	3
ƿ	Ear	2
Ɑ	Dalath	2
Ⲁ	I	2
Ⲁ	Alaph	2
Ɑ	Z	2
Ɑ	X	2
Ɑ	Nūn	2
Ɑ	Tli	2

Profile: Adam

Rysunek 3.6: Widok statystyk

3.2 Konto użytkownika

Wszystkie informacje dotyczące kont użytkowników, w tym ich postępy, wyniki i statystyki błędów, są przechowywane w bazie danych MySQL, głównie w tabeli `users`. Tworzenie użytkowników zostało poddane odpowiedniej walidacji, dzięki czemu każda nazwa profilu jest unikalna.

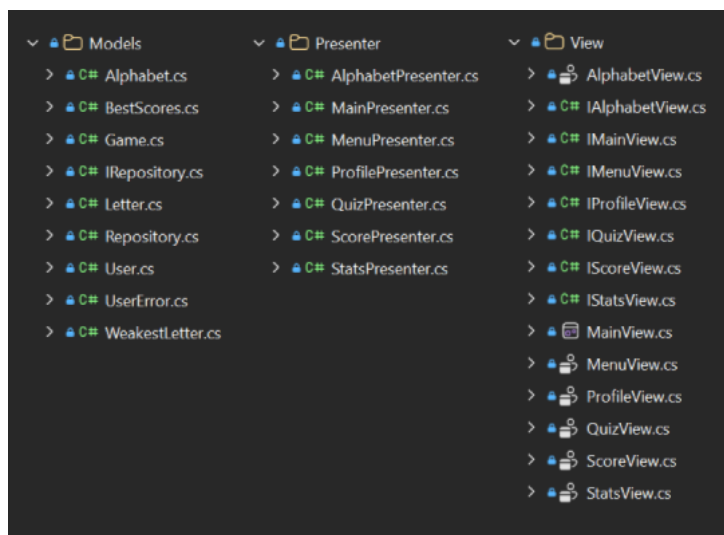
Jako, że wszystkie dane są przechowywane są tylko i wyłącznie lokalnie do profilu należy traktować bardziej jako osobne „zapisy stanu gry”, stąd decyzja o braku zabezpieczenia profili hasłami.

3.3 Podsumowanie i wnioski

Projekt spełnił wszystkie założenia projektowe, finalna baza składa się z sześciu tabel połączonych relacjami. Końcowo tabela `letters` składa się z aż 1042 rekordów, z czego każdy reprezentuje unikalną literę w danym systemie pisma.

Aplikacja skutecznie implementuje mechaniki grywalizacji takie jak system punktów życia, zapisywanie oraz wyświetlanie najlepszych wyników gracza, możliwość przeglądania statystyk oraz zwiększenie regrywalności poprzez dostosowany do gracza interfejs użytkownika.

Program zgodnie z założeniami został napisany w architekturze MVP w języku C#. Kod źródłowy składa się z 38 plików `.cs`. Za komunikację z bazą danych odpowiada plik `Repository.cs`, do którego także został przygotowany odpowiedni interfejs `IRepository.cs`. Do obsługi bazy danych została napisana specjalna procedura `BESTSCORES`, pozwalająca na bezproblemowe i bezpieczne pozyskiwanie potrzebnych informacji z bazy.



Rysunek 3.7: Pliki projektu

Oczywistym sposobem dalszego rozwoju aplikacji jest implementacja kolejnych systemów pisma, w tym tych niefonetycznych. Warto również bardziej rozwinąć mechaniki grywalizacji, np. poprzez dodanie możliwości porównywania najlepszych wyników między różnymi graczami czy też dodanie trybu „na czas”.